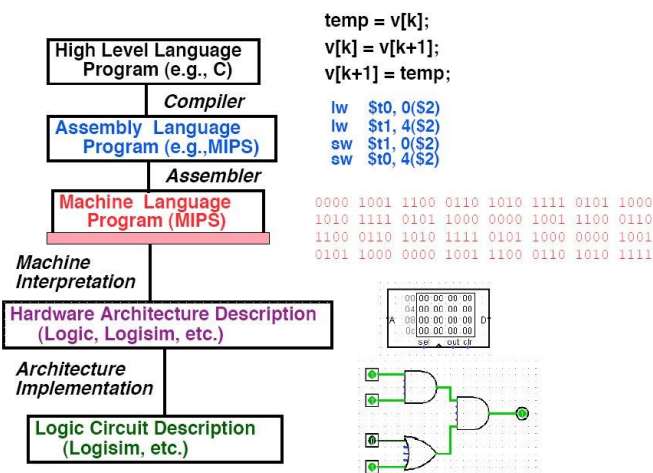


CG2271 Real Time Operating Systems

Tutorial 0 – Introduction to Assembly

SUGGESTED SOLUTIONS

Computers, being based on the binary number system, are inherently numerical, and can only understand numbers. For this reason, programs that we write in C, Python, JavaScript or even Java ultimately become a series of binary numbers. This binary encoding of programs is called “machine code” made up of numeric “machine instructions”. The diagram below illustrates this idea:



Because strings of numbers are very hard to read and understand, computer scientists produced an intermediate human-readable language called “assembly language”. Each instruction of assembly corresponds directly to a particular binary machine instruction.

Operating Systems necessarily operate on programs at the binary level, and since there is a 1-1 correspondence between binary instructions and assembly language, this tutorial aims to get you comfortable with assembly language as quickly as possible, so that the lecture materials are easier to understand. We will use MIPS assembly, which is much easier to understand than ARM.

Basic assembly instructions:

Instruction	Meaning
lw \$r1, var	Load contents of variable var into register \$r1
sw \$r1, var	Write contents of register \$r1 into variable var.
li \$r1, const	Load constant value const into \$r1. const can be negative
add \$r1, \$r2, \$r3	$\$r1 = \$r2 + \$r3$
add \$r1, \$r2, const	$\$r1 = \$r2 + \text{const.}$
sub \$r1, \$r2, \$r3	$\$r1 = \$r2 - \$r3$
mul \$r1, \$r2, \$r3	$\$r1 = \$r2 * \$r3$

mul \$r1, \$r2, const	\$r1 = \$r2 * const
div \$r1, \$r2, \$r3	\$r1 = \$r2 / \$r3
div \$r1, \$r2, const	\$r1 = \$r2 / const
slt \$r1, \$r2, \$r3	\$r1 = 1 if \$r2 < \$r3, 0 if \$r2 >= \$r3
slt \$r1, \$r2, const	\$r1 = 1 if \$r2 < const, 0 if \$r2 >= const
beq \$r1, \$r2, target	Branch to target if \$r1 == \$r2
beq \$r1, const, target	Branch to target if \$r1 == const
bne \$r1, \$r2, target	Branch to target if \$r1 != \$r2
bne \$r1, const, target	Branch to target if \$r1 != const
j target	Jump to target

General Purpose Registers:

General purpose registers range from \$0 to \$31.

Section 1. Registers and Load/Store Models

Question 1.

Using ChatGPT or other LLMs, find out what processor general purpose registers (GPRs) are, and summarize, in your own words, what it says.

General purpose registers (GPRs) are very fast one-word long storage locations within the CPU core that are used to store data and results for processing. There are limited numbers of GPRs in processors, usually from 4 to 32 of them.

Question 2.

Most processors and microcontrollers have a “load/store architecture”, meaning that program variables are always loaded from memory into a GPR, and operations are performed only on GPRs, with the results written back to the variable using “store instructions”. Some examples:

C	Assembly	Comments
$x = x + 1$	lw \$1, x add \$1, \$1, 1 sw \$1, x	# Load x into register \$1 # Increment \$1 # Store \$1 back into x
$x = y + z - 2$	lw \$1, y lw \$2, z add \$2, \$1, \$2 sub \$2, \$2, -2 sw \$2, x	# Load y into \$1 # Load z into \$2 # \$2 = \$1 + \$2 # \$2 = \$2 - 2 # x = \$2

Rewrite the following into assembly:

$x = x + 5$

lw \$1, x

add \$1, \$1, 5

sw \$1, x

$x = x - 1$

lw \$1, x

add \$1, \$1, -1

sw \$1, x

$x = x + y - z$

lw \$1, x

lw \$2, y

lw \$3, z

add \$1, \$1, \$2 # \$1 = x + y

sub \$1, \$1, \$3 # \$1 = \$1 - z = (x + y) - z = x + y - z

sw \$1, x

Section 2 Decisions

Decisions are handled using compares and branches. Some examples:

C	Assembly	Comments
<pre>if(c == 5) x = x + 1; else y = y - 1;</pre>	<pre>lw \$1, c bne \$1, 5, else lw \$2, x add \$2, \$2, 1 sw \$2, x j cont else: lw \$2, y add \$y, \$y, -1 sw \$2, y cont: ...</pre>	<pre># Load c into register \$1 # Branch to else if \$1 != 5 # IF part; load x # IF part; increment \$2 # IF part, store \$2 to x # Exit IF part # ELSE part: Load y into \$2 # ELSE part: subtract 1 from \$2 # ELSE part: Write \$2 back to y # Code after if..else.. statement</pre>
<pre>if(c < 5): x = x * 2; else: y = y / 2;</pre>	<pre>lw \$1, c slt \$1, \$1, 5 beq \$1, 0, else lw \$1, x mul \$1, \$1, 2 sw \$1, x j cont else: lw \$1, y div \$1, \$1, 2 sw \$1, y cont: ...</pre>	<pre># Logic: c<5 means we branch to # else part if c >= 5. I.e. slt \$r1, \$r2, 5 # results in \$r1==0 # IF part: Load x into \$1 # IF part: Multiply \$1 by 2 # IF part: Store \$1 to x # IF part: Jump to rest of code # ELSE part: Load y into \$1 # ELSE part: Divide \$1 by 2 # ELSE part: Store \$1 to y # Rest of code</pre>

Question 3.

Translate the following into assembly:

```
if (c >= 3)
    y = y + 1;
else:
    y = y - 1;
```



```
lw $1, c
slt $2, $1, 3
bne $2, 0, else
lw $2, y
addi $2, $2, 1
sw $2, y
j end
else: lw $2, y
      add $2, $2, -1
```

```

        sw $2, y
end:    .. rest of code ..

```

Section 3. Loops

Loops are also implemented using comparisons and branches. In the standard assembly language design pattern for while and for statements, we test for the EXIT condition and branch to the exit point of the loop, rather than the continuation condition. We test for continuation instead for do..while statements.

The table below shows how standard C loops are typically translated to assembly:

C	Assembly	Comments
<pre> x = 0; while (x < 5) y = y - 2; x++; </pre>	<pre> li \$1, 0 sw \$1, x start: slt \$2, \$1, 5 beq \$2, 0, end lw \$2, y add \$2, \$2, -2 sw \$2, y add \$1, \$1, 1 sw \$1, x j start end: </pre>	<pre> # Zero x # Exit when !(x<5), i.e. # x>=5 # Load y into \$2 # Decrement \$2 # Write back to y # Increment x # Jump back to start </pre>
<pre> for(x=0; x<5; x++) y = y - 2; </pre>	Same as above	Same as above
<pre> x = 0; do { y = y - 2; } while(x < 5); </pre>	<pre> li \$1, 0 sw \$1, x start: lw \$2, y add \$2, \$2, -2 sw \$2, y add \$1, \$1, 1 sw \$1, x slt \$3, \$1, 5 bne \$3, 0, start </pre>	<pre> # Initialize x to 0 </pre>

Notice how, for efficiency, the loop variable x is only written back to, and not read, since we always have the latest value in \$1.

Question 4.

Rewrite the following C fragment into assembly.

```
for(i=0; i<10; i++) {  
    y = y - 1;  
    if(y < 0)  
        z = z + y;  
    else  
        z = z - y;  
}
```

```
li $1, 0          # Initialize i to 0  
sw $1, i  
start: slt $2, $1, 10    # Test if i < 10  
      beq $2, 0, exit    # i>=10, exit  
      lw $2, y          # Load y  
      addi $2, $2, -1    # Subtract 1  
      sw $2, y          # Write back to y  
      slt $3, $2, 0      # Is y<0  
      beq $3, $0, else   # No, y>=0, branch to else  
      lw $3, z          # Yes, y<0. Load z  
      add $3, $3, $2     # Add y to z  
      sw $3, z          # Write back to z  
      j cont           # Jump over else part  
else:  lw $3, z          # y>=0. Load z  
      sub $3, $3, $2     # Subtract y from z  
      sw $3, z          # Write back to z  
cont:  add $1, $1, 1     # Increment i  
      sw $1, i          # Update i  
      j start          # Go for next iteration  
exit:  ... Rest of program ...
```